

CockroachDB — 4-Day Course

Outline

Distributed SQL in Practice

Course Outline

Course Objectives

By the end of the course, attendees will be able to:

- **Explain** CockroachDB's distributed SQL architecture and how it achieves survivability, consistency, and scale.
- **Master** the deployment, configuration, and day-2 operations of CockroachDB clusters from a single node to multi-region.
- **Design** schemas, optimize queries, and manage transactions in a distributed SQL environment.
- **Implement** multi-region topologies, geo-partitioning, and resilience strategies for production workloads.
- **Apply** security hardening, observability, capacity planning, and disaster recovery practices for production-ready deployments.
- **Integrate** CockroachDB with application stacks: ORMs with retry handling, outbox patterns, connection pooling at scale.
- **Migrate** from PostgreSQL using MOLT and zero-downtime cutover techniques.
- **Operate** CockroachDB on Kubernetes with the official `cockroach-operator`.

Course Length: 4 Days

Day 1 — Module 1: Architecture & Foundations

Introduction to CockroachDB

- What CockroachDB is: a distributed SQL database built for survivability
- Comparison landscape: CockroachDB vs. PostgreSQL, Spanner, YugabyteDB, TiDB
- Use cases and when CockroachDB is the right fit

Core Architecture

- Ranges, Raft consensus, leaseholders, and the distributed storage layer
- The KV layer and how SQL maps onto it
- Multi-active availability vs. traditional primary/replica models
- Node communication, gossip protocol, and cluster membership

Installation, Configuration & Tooling

- Deploying a local multi-node cluster (`cockroach start` , `cockroach demo`)
- DB Console walkthrough: metrics, jobs, statements, sessions
- CockroachDB's PostgreSQL wire-protocol compatibility — what works, what doesn't

SQL Dialect & Schema Design (*emphasis: patterns for high-volume workloads*)

- SQL dialect specifics: `SERIAL` vs. `UUID` , sequences, hash-sharded indexes
- Primary key choice as a **distribution decision**, not a uniqueness decision
- Data types, constraints, and indexing strategies
- Introducing the **Schema Patterns Playbook** that runs through Days 2-4:
 - Hash-Sharded Time-Series PK
 - Append-Only Event Log
 - Per-Tenant Co-located PK
 - Sharded Counter
 - Time-Bucketed Composite PK
 - Hot/Cold Split via Partial Index
 - GLOBAL Reference Table
 - Outbox Pattern + CDC
 - TTL Table for Automatic Expiry
 - Pre-Split + Bulk Import

Day 1 Labs

#	Lab	Duration
1	Cluster Bootstrap & Lifecycle	65 min
2	DB Console & SQL Operational Tour	60 min
3	Schema Design — Hotspots & Distribution Strategies	70 min
4	Indexing Strategies — Every Index Type That Matters	60 min

Day 2 — Module 2: Distributed Operations & Resilience

Transactions & Consistency

- Transaction model: serializable isolation by default
- MVCC, read/write intents, transaction contention and retries
- Follower reads, `AS OF SYSTEM TIME`, and stale-read trade-offs
- `SELECT FOR UPDATE` and pessimistic locking patterns

Query Optimization

- `EXPLAIN` / `EXPLAIN ANALYZE` for query analysis
- Index selection, statistics collection, cost-based optimization
- DistSQL plans, vectorized execution, statement diagnostics bundles
- Common anti-patterns: sequential keys, cross-range transactions, implicit transactions

Multi-Region Topologies & Resilience

- Replication zones and zone configurations
- Geo-partitioned tables: `REGIONAL BY ROW`, `REGIONAL BY TABLE`, `GLOBAL`
- Multi-region cluster topologies: survival goals (zone vs. region)
- Failure-domain math: replicas, quorum, and what survives what

Replication, Rebalancing & Repair

- Allocator internals and decision-making
- Node decommissioning, rebalancing, and recovery flows
- Reading and acting on under-replicated range alerts

High-Volume Schema Patterns

- **Hash-Sharded Time-Series PK** — write-heavy timestamp-ordered data without rightmost-range hotspots
- **Append-Only Event Log** — UUID PK, no updates, TTL cleanup
- **Time-Bucketed Composite PK** — `(tenant_id, hour_bucket, id)` for fast time-range queries
- **Sharded Counter** — replaces single-row `UPDATE counter SET n = n + 1`
- **Hot/Cold Split** — small partial index on "recent" rows plus full table for archives
- **Outbox Pattern** — events table written in the same transaction as business data, shipped downstream via CDC

- **Anti-patterns to avoid:** `SERIAL` on hot paths, low-cardinality lead columns, single-row queues, "index everything"

Throughput Engineering

- Bulk loading: `IMPORT INTO` vs `COPY` vs multi-row `INSERT` vs single-row inserts
- Pre-splits at scale: `ALTER TABLE ... SPLIT AT` before bulk imports
- Online schema changes: what runs as a background job and what doesn't
- Range size tuning for very-large tables
- Connection pooling for high-write workloads (PgBouncer transaction mode, HikariCP sizing)
- Measuring throughput: `cockroach workload` , p99 latency vs sustained QPS

Day 2 Labs

#	Lab	Duration
5	Transactions, Contention & Retry Loops	65 min
6	EXPLAIN ANALYZE & Query Tuning	60 min
7	Multi-Region Topologies & Survival Goals	65 min
8	Throughput Engineering — Bulk Import, Batching & Schema Pattern Playbook	60 min

Day 3 — Module 3: Production Operations & Security

Observability Stack

- Prometheus scrape targets, dashboards, alerting rules
- Grafana dashboards: built-in plus custom SLO views
- OpenTelemetry traces from CockroachDB
- Structured log channels: `SENSITIVE_ACCESS` , `USER_ADMIN` , `OPS` , `HEALTH`

Capacity Planning & Hardware Sizing (*throughput-target driven*)

- CPU, RAM, disk, and network budgets per node
- NVMe vs. network-attached storage trade-offs
- When to add nodes vs. upgrade existing ones
- Quotas, admission control, and protecting against noisy neighbors
- **Sizing exercises** by target throughput: 1k QPS, 10k QPS, 100k QPS, 1M QPS
- Storage-to-CPU ratio for write-heavy vs read-heavy workloads

Workload Simulation & Benchmarking

- The `cockroach workload` suite: KV, MovR, TPC-C, YCSB, TPC-H, bank
- Generating realistic load for sizing and regression tests
- Reading workload output: throughput vs. p99 latency vs. tail behavior

Backup, Restore & Disaster Recovery

- Full, incremental, and scheduled backups (BACKUP/RESTORE, cloud storage, userfile)
- Restore strategies and point-in-time recovery
- Cross-cluster restore and DR runbooks
- RTO / RPO math: what your backup cadence actually guarantees

Security Hardening

- TLS rotation procedures for node and client certs
- RBAC inheritance, default privileges, and least-privilege roles
- SSO / OIDC integration with an identity provider
- Audit logging end to end: enabling, capturing, shipping to a SIEM
- Compliance considerations: SOC2, GDPR, HIPAA

On-Call Practices

- `cockroach debug zip` and what to capture during an incident

- Statement diagnostics bundles and how to read them
- Runbook structure and escalation paths

Day 3 Labs

#	Lab	Duration
9	Build a Prometheus + Grafana Observability Stack with SLO Dashboards	60 min
10	TPC-C Benchmark & Capacity Sizing Exercise	65 min
11	BACKUP / RESTORE / Schedules & Cross-Cluster DR Drill	65 min
12	Security Hardening End-to-End — TLS Rotation, SSO, Audit Pipeline	55 min

Day 4 — Module 4: Advanced Features & Real-World Integration

Change Data Capture in Depth

- Sinks: Kafka, webhook, cloud storage, Google Pub/Sub
- Schema registry integration (Confluent / Apicurio)
- Exactly-once delivery semantics and how to configure them
- Resolved timestamps and the downstream frontier problem
- Building idempotent downstream consumers

Application Integration Patterns

- Outbox pattern using CockroachDB + CDC
- Idempotency keys and request-replay safety
- Optimistic concurrency control vs. `SELECT FOR UPDATE`
- Connection pooling at scale: PgBouncer modes, HikariCP, sqlx pools

ORM & Driver Integration

- Python: psycopg, SQLAlchemy with crdb retry helper
- Go: pgx, GORM, sqlx with crdb-go
- Node.js: node-postgres, Prisma, Knex
- Java/Kotlin: pgjdbc, Hibernate (CRDB dialect), jOOQ

Schema Migrations & CI/CD

- Migration tools: Flyway, Liquibase, Atlas, golang-migrate, Alembic
- Coordinating online schema changes with rolling application deploys
- Pre-deploy backup hygiene and rollback planning

Migrating from PostgreSQL

- MOLT verifier and the dialect differences cheat sheet
- Zero-downtime cutover patterns: logical replication, dual-write, shadow reads
- Data movement: `IMPORT INTO`, AWS DMS, custom pipelines
- When NOT to migrate: workloads PostgreSQL still does better

Kubernetes Deployment

- `cockroach-operator` and CockroachDB CRDs
- Helm charts and cert-manager integration

- Pod sizing, anti-affinity, and region-aware StatefulSets
- Logical multi-tenancy on a shared Kubernetes cluster

Advanced Features Survey

- TTL (time-to-live) tables for automatic data expiry
- User-defined functions (UDFs)
- Spatial data types and queries
- Vector indexes (v23.2+) for embeddings and AI workloads
- Full-text search

Cost & Deployment Models

- CockroachDB Serverless vs. Dedicated vs. Self-Hosted — when to use what
- Enterprise vs. Core licensing — what changes
- Predicting costs at the workload and team scale

Day 4 Labs

#	Lab	Duration
13	CDC → Kafka → Downstream Consumer with Resolved-Timestamp Frontier	55 min
14	Outbox Pattern + Idempotent Retry Loop in Python/Go	55 min
15	Migrate a PostgreSQL Schema and Live Data with MOLT	55 min
16	Deploy CockroachDB on Kubernetes with cockroach-operator and kind	70 min

Pacing Summary

Day	Lab Time	Lecture Time	Total
Day 1	255 min	~45 min	6 hrs
Day 2	250 min	~45 min	6 hrs
Day 3	245 min	~45 min	6 hrs
Day 4	235 min	~45 min	6 hrs
Total	985 min	~180 min	24 hrs

Lab share: ~68% Lecture share: ~12% Breaks and buffer: ~20%

Class days are Monday–Thursday, six hours each. Every lab has a core set of parts sized to the figures above plus an *Optional — If Time Allows* section; each deck ends with an *Appendix — If Time Allows* stack of deep-dive slides. Neither is needed to follow the course.

Infrastructure Notes

- **Labs 1–12 and 14** run against `cockroach demo` or local `cockroach start` clusters — same prerequisites as the 2-day course.
 - **Lab 13** (CDC → Kafka) needs a Kafka broker. A small `docker-compose.yml` ships a single-broker Kafka alongside the lab.
 - **Lab 15** (MOLT migration) needs a PostgreSQL instance as the source. A `docker-compose.yml` snippet provisions one.
 - **Lab 16** (Kubernetes) needs `kind` or `minikube` on the learner's machine. The containerized test runner uses `kind` inside the existing image with `--privileged`, or a sibling-DinD setup.
-

Appendix A: Schema Patterns Playbook (*takeaway reference*)

Named patterns introduced in Day 1, applied across Days 2–4, and reinforced in Lab 8. Each pattern lists its target workload, the canonical schema, and the trap it avoids.

1. Hash-Sharded Time-Series PK

For: write-heavy, time-ordered data (events, metrics, audit logs)

```
CREATE TABLE events (  
  account_id  UUID,  
  created     TIMESTAMPTZ DEFAULT now(),  
  id          UUID DEFAULT gen_random_uuid(), -- per-row tiebreaker: now() is  
  payload     STRING,                        -- the TRANSACTION timestamp  
  PRIMARY KEY (account_id, created, id) USING HASH WITH (bucket_count = 16)  
);
```

Avoids: rightmost-range write hotspot on monotonic keys.

2. Append-Only Event Log

For: ingest-heavy logs that are never updated and aged out by time

```
CREATE TABLE event_log (  
  id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  ts      TIMESTAMPTZ DEFAULT now(),  
  payload JSONB  
) WITH (ttl_expire_after = '30 days', ttl_job_cron = '@hourly');
```

Avoids: unbounded growth; DELETE-as-you-go jobs.

3. Per-Tenant Co-located PK

For: multi-tenant SaaS where most queries filter by tenant

```
CREATE TABLE orders (  
  tenant_id UUID,  
  order_id  UUID DEFAULT gen_random_uuid(),  
  ...  
  PRIMARY KEY (tenant_id, order_id)  
);
```

Avoids: cross-range scans for single-tenant queries. **Watch:** one hot tenant becomes one hot range — combine with hash sharding for a hot tenant.

4. Time-Bucketed Composite PK

For: per-tenant time-range queries that need ordering

```
CREATE TABLE metrics (  
  tenant_id  UUID,  
  hour_bucket TIMESTAMPTZ, -- truncated to hour  
  metric_id  UUID DEFAULT gen_random_uuid(),  
  value      DECIMAL,  
  PRIMARY KEY (tenant_id, hour_bucket, metric_id)  
);
```

Avoids: unbounded range scans; sequential keys.

5. Sharded Counter

For: very-high-frequency counters

```

CREATE TABLE counter_shards (
  name STRING, shard INT, n INT DEFAULT 0,
  PRIMARY KEY (name, shard)
);
-- Increment: the APPLICATION picks the shard (random() in a predicate is
-- evaluated per row and silently loses or duplicates increments)
UPDATE counter_shards SET n = n + 1
WHERE name = 'page_views' AND shard = $1; -- $1 = randint(0, 15)
-- Read: sum across shards
SELECT sum(n) FROM counter_shards WHERE name = 'page_views';

```

Avoids: SQLSTATE 40001 retry storms on single-row counters.

6. Hot/Cold Split via Partial Index

For: big table where 99% of queries hit a small recent slice

```

CREATE INDEX orders_open ON orders(total DESC)
  STORING (customer_id) WHERE status = 'open';

```

Avoids: wasting index space on rows that are never queried.

7. GLOBAL Reference Table

For: small read-anywhere lookup data (countries, currencies, feature flags)

```

CREATE TABLE country_codes (code STRING PRIMARY KEY, name STRING) LOCALITY GLOBAL;

```

Avoids: cross-region reads on hot lookup paths. **Watch:** writes are slow.

8. Outbox Pattern + CDC

For: publishing events atomically with business writes

```

CREATE TABLE events_outbox (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  topic       STRING NOT NULL,
  payload     JSONB,
  created     TIMESTAMPTZ DEFAULT now()
) WITH (ttl_expire_after = '7 days');

CREATE CHANGEFEED FOR TABLE events_outbox INTO 'kafka://...';

```

Avoids: dual-write inconsistency between database and message queue.

9. TTL Table

For: rows with a natural expiration date (sessions, tokens, soft locks)

```
CREATE TABLE sessions (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id    UUID,  
  expires     TIMESTAMPTZ  
) WITH (ttl_expiration_expression = 'expires', ttl_job_cron = '* / 10 * * * *');
```

Avoids: background DELETE jobs that compete with live traffic.

10. Pre-Split + Bulk Import

For: one-time loads of sequential-key data (analytics imports, archive restores)

```
ALTER TABLE big_import SPLIT AT  
  VALUES (100000), (200000), (300000), ..., (900000);  
  
IMPORT INTO big_import (id, payload)  
CSV DATA ('s3://...');
```

Avoids: writing 100M rows into a single range, then waiting for the splitter to catch up.

Anti-Patterns to Spot in Code Review

- `id SERIAL PRIMARY KEY` on a high-volume table → use UUID or hash-sharded
- `UPDATE counters SET n = n + 1 WHERE id = '...'` → use Sharded Counter
- Low-cardinality lead column in PK (`region` , `status` , ...) → reorder or rethink
- Indexes on every column "just in case" → write amplification multiplies inserts
- `DELETE FROM logs WHERE created < ...` as a recurring job → use TTL Table
- Application reads then writes without `BEGIN ... COMMIT` → no atomicity guarantees
- Writing to Kafka **and** database from app code → use Outbox + CDC instead